SIMATS ENGINEERING

ASSESSMENT TOOL 2: Industry Problem-Based Assignment

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS **COURSE CODE: MLA01**

COURSE OUTCOME COVERED

CO 5: *Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.*(BTL 3)

Assessment Tool Weightage: 40%

Total Marks: 100 **Time: 90 Mins**

No. of Questions: 1

Annexure A

Industry Problem-Based Assignment

Code of Conduct:

I P Surendra Kumar Reddy (192425224) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:P Surendra Kumar Reddy

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Acquisition and Knowledge Representation	10%	
3. Production Rule / Knowledge Base Design	15%	
4. Prolog Implementation and Programming	15%	
5. Forward Chaining Implementation and Demonstration	10%	
6. Backward Chaining Implementation and Demonstration	10%	
7. Procedural and Non-Procedural Paradigm Analysis	10%	
8. Unification, Backtracking and Inference Accuracy	10%	
9. Testing, Results and System Demonstration	5%	
10. Documentation, GitHub Repository and Technical Presentation	5%	
Total	**100%**	

Signature of the Faculty Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING

S.No.	Industry Domain	Real-World Problem Statement
1	**Automobile Industry**	A vehicle service center receives customer complaints such as engine overheating, difficulty starting, abnormal noise, low mileage, and warning-light activation. Develop a Prolog-based expert system that identifies probable vehicle faults and recommends appropriate diagnostic actions using production rules and logical inference.
2	**Healthcare**	A hospital wants to assist preliminary diagnosis by analyzing symptoms such as fever, cough, breathing difficulty, body pain, and fatigue. Develop a rule-based diagnostic system that identifies possible conditions and explains the reasoning behind its conclusions using Prolog.
3	**Banking**	A bank needs an automated loan pre-screening system that evaluates customer income, credit score, employment status, existing loans, and repayment history. Develop a Prolog-based decision-support system that determines loan eligibility and provides reasons for the decision.
4	**Cybersecurity**	A Security Operations Center receives events such as repeated failed logins, unusual login locations, privilege escalation, suspicious file access, and abnormal network traffic. Develop an expert system that identifies possible security threats and recommends appropriate actions.
5	**Manufacturing**	A manufacturing plant experiences unexpected machine failures. Operators provide observations such as abnormal vibration, excessive temperature, unusual noise, pressure changes, and reduced production speed. Develop a machine-fault diagnosis expert system that identifies probable faults and suggests maintenance actions.
6	**Agriculture**	Farmers observe symptoms such as yellow leaves, spots, wilting, poor growth, and pest infestation. Develop a crop-disease advisory expert system that identifies possible diseases based on crop, soil, weather, and visible symptoms and recommends suitable actions.
7	**Telecommunications**	A telecom provider receives complaints about slow internet, frequent call drops, poor signal strength, and network unavailability. Develop an expert troubleshooting system that identifies probable network problems and recommends troubleshooting steps.
8	**E-Commerce**	An online retailer wants to recommend products based on customer preferences, purchase history, budget, product category, and previous interactions. Develop a rule-based recommendation system that generates personalized recommendations and explains why each product was recommended.

9	**Power & Energy**	A power distribution company monitors voltage, current, frequency, temperature, and load conditions. Develop an expert system for fault diagnosis that identifies possible overload, voltage instability, equipment failure, or abnormal operating conditions and recommends corrective actions.
10	**Human Resources**	An organization wants to identify suitable training programs for employees based on job role, technical skills, performance, experience, and competency gaps. Develop a rule-based employee training recommendation system that recommends appropriate training and explains the reasoning.

Structure of the Report:

S.No.	Report Section	Expected Content
1	Title, Abstract & Introduction	Project title, brief abstract, industry background, problem context, and importance of the selected problem.
2	Problem Statement & Objectives	Clearly define the real-world industry problem, inputs, expected outputs, stakeholders, and specific objectives.
3	Domain Analysis & Knowledge Acquisition	Identify entities, facts, conditions, decisions, and explain how domain knowledge was collected from reliable sources.
4	Knowledge Representation & Production Rules	Represent domain knowledge using Prolog facts, predicates, and production rules.
5	Expert System Architecture	Provide and explain the architecture showing the user, knowledge base, inference engine, and output/explanation module.
6	Inference Mechanism & Algorithms	Explain and demonstrate forward chaining, backward chaining, unification, and backtracking with suitable algorithms/pseudocode.
7	Prolog Implementation	Describe the SWI-Prolog implementation, important predicates/rules, queries, and provide the GitHub repository link.
8	Test Cases & Results	Provide minimum 5 industry-based test cases, queries, expected outputs, actual outputs, and inference traces.
9	Analysis, Limitations & Future Enhancements	Analyze system effectiveness and reasoning accuracy; discuss limitations and propose future improvements.
10	Conclusion & References	Summarize the developed expert system, major findings, industry relevance, and provide properly formatted references.

INTELLIGENT LOAN PRE-SCREENING SYSTEM USING PROLOG

Abstract

This project presents a Prolog-Based Loan Pre-Screening Expert System developed for the banking domain. The system evaluates customer loan eligibility using factors such as income, credit score, employment status, existing loan status, and repayment history. Customer information is represented as Prolog facts, while loan eligibility criteria are represented using production rules. The system demonstrates important Artificial Intelligence and Prolog concepts, including knowledge representation, logical inference, forward chaining, backward chaining, unification, and backtracking. The system was implemented and tested using five different customer cases to evaluate its reasoning accuracy and consistency. An explanation facility is also included to provide the reasoning behind the final loan eligibility decision. The proposed system demonstrates how a rule-based expert system can support preliminary loan screening in the banking industry.

1. Introduction

Banks receive many loan applications and must evaluate several factors before making an initial decision. Important factors may include the applicant's income, credit score, employment status, existing loans, and repayment history.

This project develops a simple **Prolog-based Loan Pre-Screening Expert System**. The system uses a knowledge base containing customer information and a set of production rules to determine whether an applicant is eligible for a loan.

The system demonstrates important Artificial Intelligence and Prolog concepts, including:

- Knowledge representation
- Facts and rules
- Production rules
- Forward chaining
- Backward chaining
- Unification
- Backtracking

- Logical inference
- Explanation of decisions

The purpose of the system is to provide an initial decision-support mechanism. It does not replace a real bank's complete loan approval process but demonstrates how an expert system can use predefined knowledge and rules to make logical decisions.

2. Problem Statement

Loan eligibility decisions involve the evaluation of multiple applicant characteristics. Performing an initial assessment manually can require repeated checking of customer information against predefined eligibility conditions.

The problem addressed in this project is to develop an intelligent decision-support system that can evaluate a customer's loan application based on the following factors:

- Income
- Credit score
- Employment status
- Existing loan status
- Repayment history

The system should use Prolog facts and rules to determine whether the customer is eligible for a loan. It should also demonstrate both forward chaining and backward chaining and provide an explanation of the final decision.

Stakeholders, Inputs and Expected Outputs

Stakeholders

The main stakeholders of the proposed system are:

- **Bank Loan Officers:** They can use the system as a preliminary decision-support tool.
- **Banking Institutions:** The system can help apply loan screening rules consistently.
- **Loan Applicants:** Applicants receive a preliminary eligibility decision based on the defined conditions.

Inputs

The system accepts the following customer information:

- Income level
- Credit score
- Employment status
- Existing loan status
- Repayment history

Expected Output

The system produces one of the following decisions:

- **LOAN ELIGIBLE**
- **LOAN NOT ELIGIBLE**

The system can also display the reasoning behind the decision by showing which conditions were satisfied or failed.

Constraints

The current system is a simplified academic prototype. It uses categorical values such as high, medium, low, good, and poor rather than actual banking data or numerical financial values.

3. Objectives

The main objectives of this project are:

1. To develop a Prolog-based expert system for loan pre-screening.
2. To represent banking-related customer information using Prolog facts.
3. To create production rules for evaluating loan eligibility.
4. To demonstrate forward chaining using available customer facts.
5. To demonstrate backward chaining by attempting to prove the goal loan_eligible(Customer).
6. To demonstrate Prolog concepts such as unification and backtracking.
7. To provide an explanation for the loan decision.
8. To test the system using at least five customer cases.

4. Domain Analysis and Knowledge Acquisition

The selected application domain is **banking**, specifically the preliminary evaluation of loan applications.For this project, the knowledge required for the system was represented using five main customer attributes:

Attribute	Description
Income	Indicates the customer's income level
Credit Score	Indicates the customer's credit quality
Employment	Indicates whether employment is stable or unstable
Existing Loan	Indicates whether the customer already has an existing loan
Repayment History	Indicates the customer's previous repayment performance

The customer information is stored in the following Prolog format:

customer(Name, Income, CreditScore, Employment, ExistingLoan, RepaymentHistory).

The following customer records were used in the knowledge base:

customer(suri, high, excellent, stable, no, good).
customer(ravi, medium, good, stable, no, good).
customer(rahul, low, poor, unstable, yes, poor).
customer(priya, high, good, stable, yes, good).
customer(anu, medium, poor, stable, no, average).

These facts form the initial knowledge base of the expert system.

Knowledge Acquisition

The knowledge used in this project was obtained by analysing common factors involved in preliminary loan assessment, such as income, creditworthiness, employment stability, existing financial obligations, and repayment history.

For the purpose of this academic project, simplified eligibility conditions were defined and represented as Prolog facts and rules. These conditions are used to demonstrate how domain knowledge can be converted into a rule-based expert system.

The knowledge acquisition process involved:

1. Identifying important customer attributes.
2. Defining possible values for each attribute.
3. Identifying relationships between customer conditions and loan eligibility.
4. Converting the identified knowledge into Prolog facts.
5. Developing production rules for logical inference.

This approach separates the **knowledge base** from the **inference mechanism**, making the system easier to modify and maintain.

5. Knowledge Representation

The knowledge base is represented using Prolog facts and rules.

5.1 Customer Facts

Each customer is represented using the predicate:

customer(Name, Income, CreditScore, Employment, ExistingLoan, RepaymentHistory).

For example:

customer(suri, high, excellent, stable, no, good).

This fact represents a customer named suri with:

- High income
- Excellent credit score
- Stable employment
- No existing loan
- Good repayment history

```
102 ?- customer(suri, Income, Credit, Employment, Loan, Repayment).
Income = high,
Credit = excellent,
Employment = stable,
Loan = no,
Repayment = good.
```

Figure 1: Knowledge Base Facts Retrieved Using Prolog Unification

6. Production Rules

The expert system uses production rules to evaluate different eligibility conditions.

Rule 1: Income Condition

A customer satisfies the income condition if the income level is high.

```
income_ok(C) :-
   customer(C, high, _, _, _, _).
```

Rule 2: Credit Score Condition

A customer satisfies the credit condition if the credit score is excellent or good.

```
credit_ok(C) :-
   customer(C, _, excellent, _, _, _).


credit_ok(C) :-
   customer(C, _, good, _, _, _).
```

Rule 3: Employment Condition

A customer satisfies the employment condition if the employment status is stable.

```
employment_ok(C) :-
   customer(C, _, _, stable, _, _).
```

Rule 4: Repayment History Condition

A customer satisfies the repayment condition if the repayment history is good.

```
repayment_ok(C) :-
   customer(C, _, _, _, _, good).
```

Rule 5: Existing Loan Condition

A customer satisfies this condition if there is no existing loan.

```
loan_status_ok(C) :-
```

customer(C, _, _, _, no, _).

Final Eligibility Rule

A customer is considered eligible when all required conditions are satisfied.

```
loan_eligible(C) :-
    income_ok(C),
    credit_ok(C),
    employment_ok(C),
    repayment_ok(C),
    loan_status_ok(C).
```

Thus, the system follows the following logical structure:

IF

- Income is high
- AND credit score is good or excellent
- AND employment is stable
- AND repayment history is good
- AND there is no existing loan

THEN

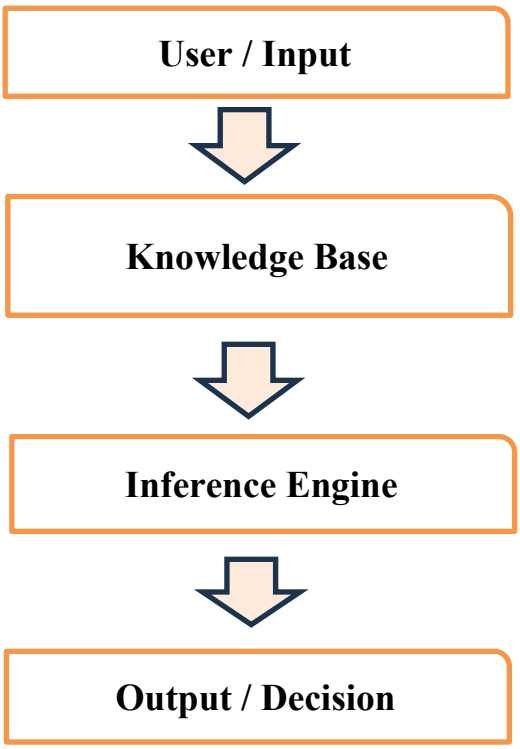
- The customer is eligible for the loan.

7. Expert System Architecture

The proposed expert system consists of four main components:

1. **User/Input**
2. **Knowledge Base**
3. **Inference Engine**
4. **Output and Explanation Module**

The overall process is shown below:



The knowledge base stores the customer facts and loan eligibility rules. The inference engine processes this knowledge and determines whether the eligibility conditions are satisfied. The final output provides the loan decision and reasoning.

8. Inference Algorithms / Pseudocode

Forward Chaining Algorithm:

START

Input customer information

Retrieve:

Income

Credit Score

Employment

Existing Loan

Repayment History

IF income condition is satisfied

THEN mark income as valid

IF credit score condition is satisfied

THEN mark credit score as valid

IF employment condition is satisfied

THEN mark employment as valid

IF repayment history condition is satisfied

THEN mark repayment history as valid

IF existing loan condition is satisfied

THEN mark loan status as valid

IF all required conditions are valid

THEN

Decision = LOAN ELIGIBLE

ELSE

Decision = LOAN NOT ELIGIBLE

Display final decision

STOP

Backward Chaining Algorithm

START

Set Goal = loan_eligible(Customer)

Check whether:

income_ok(Customer)

Check whether:

credit_ok(Customer)

Check whether:

employment_ok(Customer)

Check whether:

repayment_ok(Customer)

Check whether:

loan_status_ok(Customer)

IF all sub-goals are proven

THEN Goal is TRUE

Decision = LOAN ELIGIBLE

ELSE

Goal cannot be proven

Decision = LOAN NOT ELIGIBLE

STOP

Unification and Backtracking Algorithm

START

Query customer(Name, Income, Credit,

Employment, Loan, Repayment)

Match variables with the first customer fact

Display the solution

IF another solution is requested

THEN backtrack to the knowledge base

Find the next matching customer fact

Repeat until no more solutions exist

STOP

9. Prolog Implementation

The system was implemented using **SWI-Prolog**.

The implementation contains:

- Customer facts
- Eligibility rules
- Forward chaining demonstration
- Backward chaining demonstration
- Explanation predicates
- Test cases
- Unification and backtracking demonstration

The program was successfully loaded into SWI-Prolog.

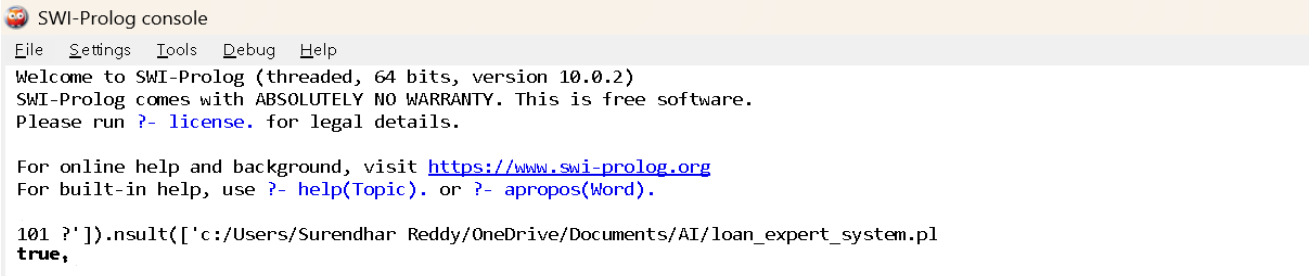


Figure 2: Successful Loading of the Prolog Loan Expert System

10. Forward Chaining

Forward chaining is a data-driven reasoning approach. The inference process starts with the available facts and applies relevant rules until a conclusion is reached.

In this system, the customer's information is first retrieved. The system then checks each eligibility condition in sequence.

For example, for customer suri, the system evaluates:

- Income = high
- Credit score = excellent
- Employment = stable
- Existing loan = no
- Repayment history = good

Since all required conditions are satisfied, the final decision is:

LOAN ELIGIBLE

```
103 ?- test1.

========================================
        FORWARD CHAINING
========================================
Customer: suri

Initial Facts:
Income = high
Credit Score = excellent
Employment = stable
Existing Loan = no
Repayment History = good

[RULE 1] Income condition satisfied.
[RULE 2] Credit score condition satisfied.
[RULE 3] Employment condition satisfied.
[RULE 4] Repayment history condition satisfied.
[RULE 5] Existing loan condition satisfied.
----------------------------------------
FINAL DECISION: LOAN ELIGIBLE
----------------------------------------
true
```

Figure 3: Forward Chaining for Loan Eligibility – Eligible Applicant

The system also demonstrates a rejected case. For customer rahul, the income condition is not satisfied because the income is low. Therefore, the system concludes:

LOAN NOT ELIGIBLE

```
104 ?- test3.

========================================
        FORWARD CHAINING
========================================
Customer: rahul

Initial Facts:
Income = low
Credit Score = poor
Employment = unstable
Existing Loan = yes
Repayment History = poor

[RULE FAILED] Income condition not satisfied.
----------------------------------------
FINAL DECISION: LOAN NOT ELIGIBLE
----------------------------------------
true .
```

Figure 4: Forward Chaining for Loan Eligibility – Rejected Applicant

This demonstrates how the system can handle both successful and unsuccessful rule evaluations.

11. Backward Chaining

Backward chaining is a goal-driven reasoning approach. Instead of starting with all facts, the system starts with a goal and attempts to prove whether the goal is true.

The main goal in this system is:

loan_eligible(Customer)

To prove this goal, Prolog checks the required sub-goals:

1. Is the income condition satisfied?
2. Is the credit score condition satisfied?
3. Is the employment condition satisfied?
4. Is the repayment condition satisfied?
5. Is the existing loan condition satisfied?

For customer suri, all sub-goals are satisfied. Therefore, the goal loan_eligible(suri) is successfully proven.

```
105 ?- backward_loan_decision(suri).

========================================
        BACKWARD CHAINING
========================================
Customer: suri

Goal: loan_eligible(suri)

Goal satisfied.
----------------------------------------
FINAL DECISION: LOAN ELIGIBLE
----------------------------------------
true.
```

Figure 5: Backward Chaining for Loan Eligibility – Eligible Applicant

For customer rahul, the required conditions cannot all be satisfied. Therefore, the goal cannot be proven.

```
106 ?- backward_loan_decision(rahul).

========================================
        BACKWARD CHAINING
========================================
Customer: rahul

Goal: loan_eligible(rahul)

Goal cannot be satisfied.
----------------------------------------
FINAL DECISION: LOAN NOT ELIGIBLE
----------------------------------------
true.
```

Figure 6: Backward Chaining for Loan Eligibility – Rejected Applicant

12. Explanation Facility

An important feature of an expert system is its ability to explain how a decision was reached.

The explain(Customer) predicate displays the values of the customer's attributes and explains whether each requirement is satisfied.

For example, the system can provide the following reasoning:

- Income requirement satisfied.
- Credit score requirement satisfied.

- Employment requirement satisfied.
- Repayment history is satisfactory.
- No existing loan condition satisfied.

The system then provides the final conclusion.

```
107 ?- explain(suri).

============================================
          LOAN DECISION EXPLANATION
============================================
Customer: suri

Income: high
Credit Score: excellent
Employment: stable
Existing Loan: no
Repayment History: good

Reasoning:
- Income requirement satisfied.
- Credit score requirement satisfied.
- Employment requirement satisfied.
- Repayment history is satisfactory.
- No existing loan condition satisfied.

Conclusion: Customer is ELIGIBLE for the loan.
true .
```

Figure 7: Explanation of Loan Eligibility Decision

This explanation facility improves the transparency of the decision-making process.

13. Test Cases and Results

The system was tested using five different customer cases.

Test Case	Customer	Query	Expected Output	Actual Output	Inference Summary
TC1	Suri	test1.	Loan Eligible	Loan Eligible	All eligibility conditions satisfied
TC2	Ravi	test2.	Loan Not Eligible	Loan Not Eligible	Income condition not satisfied under current rules
TC3	Rahul	test3.	Loan Not Eligible	Loan Not Eligible	Income and other conditions fail
TC4	Priya	test4.	Loan Not Eligible	Loan Not Eligible	Existing loan condition fails
TC5	Anu	test5.	Loan Not Eligible	Loan Not Eligible	Income, credit, and repayment conditions fail

The results are based on the current implementation rules.

For example:

- **Suri** satisfies all eligibility conditions.
- **Ravi** does not satisfy the current high-income requirement.
- **Rahul** does not satisfy the income condition and also has several other unfavorable conditions.
- **Priya** has an existing loan, which prevents eligibility under the implemented rules.
- **Anu** does not satisfy the income, credit, and repayment requirements.

Test Cases

Test Case 1: Suri

```
108 ?- test1.

========================================
        FORWARD CHAINING
========================================
Customer: suri

Initial Facts:
Income = high
Credit Score = excellent
Employment = stable
Existing Loan = no
Repayment History = good

[RULE 1] Income condition satisfied.
[RULE 2] Credit score condition satisfied.
[RULE 3] Employment condition satisfied.
[RULE 4] Repayment history condition satisfied.
[RULE 5] Existing loan condition satisfied.
----------------------------------------
FINAL DECISION: LOAN ELIGIBLE
----------------------------------------
true .
```

Figure 8: Test Cases and Results of the Loan Eligibility Expert System for Suri

Test Case 2: Ravi

```
109 ?- test2.

========================================
        FORWARD CHAINING
========================================
Customer: ravi

Initial Facts:
Income = medium
Credit Score = good
Employment = stable
Existing Loan = no
Repayment History = good

[RULE FAILED] Income condition not satisfied.
----------------------------------------
FINAL DECISION: LOAN NOT ELIGIBLE
----------------------------------------
true .
```

Figure 9: Test Case and Results of the Loan Eligibility Expert System for Ravi

Test Case 3: Rahul

```
110 ?- test3.

========================================
        FORWARD CHAINING
========================================
Customer: rahul

Initial Facts:
Income = low
Credit Score = poor
Employment = unstable
Existing Loan = yes
Repayment History = poor

[RULE FAILED] Income condition not satisfied.
----------------------------------------
FINAL DECISION: LOAN NOT ELIGIBLE
----------------------------------------
true .
```

Figure 10: Test Cases and Results of the Loan Eligibility Expert System for Rahul

Test Case 4: Priya

```
111 ?- test4.

========================================
        FORWARD CHAINING
========================================
Customer: priya

Initial Facts:
Income = high
Credit Score = good
Employment = stable
Existing Loan = yes
Repayment History = good

[RULE 1] Income condition satisfied.
[RULE 2] Credit score condition satisfied.
[RULE 3] Employment condition satisfied.
[RULE 4] Repayment history condition satisfied.
[RULE FAILED] Existing loan condition not satisfied.
----------------------------------------
FINAL DECISION: LOAN NOT ELIGIBLE
----------------------------------------
true.
```

Figure 11: Test Cases and Results of the Loan Eligibility Expert System for Priya

Test Case 5: Anu

```
112 ?- test5.

========================================
        FORWARD CHAINING
========================================
Customer: anu

Initial Facts:
Income = medium
Credit Score = poor
Employment = stable
Existing Loan = no
Repayment History = average

[RULE FAILED] Income condition not satisfied.
----------------------------------------
FINAL DECISION: LOAN NOT ELIGIBLE
----------------------------------------
true .
```

Figure 12: Test Cases and Results of the Loan Eligibility Expert System for Anu

The test cases demonstrate that the system can correctly apply the implemented rules to different customer profiles.

14. Unification and Backtracking

14.1 Unification

Unification is an important feature of Prolog. It allows variables to be matched with values stored in facts.

For example, the following query:

customer(Name, Income, Credit, Employment, Loan, Repayment).

causes Prolog to match the variables with the values stored in the customer facts.

The variable Name can be unified with values such as:

suri
ravi
rahul
priya
anu

Similarly, the other variables are matched with the corresponding customer information.

14.2 Backtracking

Backtracking allows Prolog to search for alternative solutions.

After the first customer fact is returned, the user can request another solution using ;. Prolog then searches the knowledge base for another fact that satisfies the query.

The query therefore produces multiple solutions for different customers.

```
113 ?- customer(Name, Income, Credit, Employment, Loan, Repayment).
Name = suri,
Income = high,
Credit = excellent,
Employment = stable,
Loan = no,
Repayment = good ;
Name = ravi,
Income = medium,
Credit = Repayment, Repayment = good,
Employment = stable,
Loan = no ;
Name = rahul,
Income = low,
Credit = Repayment, Repayment = poor,
Employment = unstable,
Loan = yes ;
Name = priya,
Income = high,
Credit = Repayment, Repayment = good,
Employment = stable,
Loan = yes ;
Name = anu,
Income = medium,
Credit = poor,
Employment = stable,
Loan = no,
Repayment = average.
```

Figure 13: Demonstration of Unification and Backtracking in the Prolog Loan Eligibility Expert System

This demonstrates the declarative nature of Prolog and its built-in search mechanism.

15. Procedural and Non-Procedural Programming Paradigms

15.1 Procedural Programming

Procedural programming focuses on specifying the sequence of steps required to solve a problem.

A procedural implementation of loan eligibility might follow a sequence such as:

1. Read customer information.
2. Check income.
3. Check credit score.
4. Check employment status.
5. Check repayment history.
6. Check existing loans.
7. Produce the final decision.

The programmer explicitly controls the sequence of operations.

15.2 Non-Procedural Programming

Prolog mainly follows a declarative or non-procedural approach.

Instead of explicitly describing every step of the solution, the programmer defines:

- Facts
- Rules
- Relationships

For example:

```
loan_eligible(C) :-
    income_ok(C),
    credit_ok(C),
    employment_ok(C),
    repayment_ok(C),
    loan_status_ok(C).
```

This rule describes the conditions under which a customer is eligible. Prolog's inference mechanism determines how to search for a solution.

15.3 Comparison

Procedural Approach	Non-Procedural Prolog Approach
Focuses on how to solve the problem	Focuses on what conditions must be true
Explicit sequence of instructions	Facts and rules define relationships
Programmer controls execution flow	Prolog uses inference and backtracking
Usually uses loops and conditional statements	Uses predicates, unification, and logical inference

The loan pre-screening system demonstrates the non-procedural approach because eligibility is represented using logical relationships and Prolog determines whether the required conditions can be satisfied.

16. Advantages of the System

The proposed expert system has several advantages:

1. **Consistency** – The same rules are applied to every customer.
2. **Automation** – Initial loan screening can be performed automatically.
3. **Transparency** – The explanation module shows the reasoning behind the decision.

4. **Easy Knowledge Representation** – Customer information can be represented using simple Prolog facts.
5. **Logical Inference** – Prolog supports unification, backtracking, and goal-based reasoning.
6. **Easy Modification** – New rules or conditions can be added to the knowledge base.

17. Analysis and Interpretation of Results

The test results demonstrate that the system consistently applies the same eligibility rules to each customer. The eligible case, represented by Suri, satisfied all required conditions, resulting in a **LOAN ELIGIBLE** decision.

The remaining test cases demonstrated different reasons for rejection. Ravi did not satisfy the high-income requirement defined in the current knowledge base. Rahul failed multiple conditions, including income and employment stability. Priya had an existing loan, while Anu did not satisfy several required conditions.

The forward chaining tests demonstrated data-driven reasoning, where the system started with customer facts and applied rules to reach a decision. The backward chaining tests demonstrated goal-driven reasoning by attempting to prove whether loan_eligible(Customer) could be satisfied.

The results indicate that the system provides consistent decisions according to the defined rules. However, the accuracy of the decision depends on the quality and completeness of the knowledge base and rules.

18. Limitations

Although the system demonstrates the main concepts of an expert system, it has several limitations.

1. The system uses a small number of customer records.
2. Income is represented using simple categories such as high, medium, and low.
3. The credit score is represented using qualitative values rather than real numerical scores.
4. The eligibility rules are simplified for demonstration purposes.
5. The system does not calculate the maximum loan amount.
6. The system does not consider factors such as age, assets, loan purpose, or debt-to-income ratio.
7. The system is intended for preliminary decision support and not for a complete real-world banking approval process.

19. Future Enhancements

The system can be improved in several ways.

1. Add more customer attributes.
2. Use numerical income and credit score values.
3. Include different types of loans.
4. Calculate a recommended loan amount.
5. Add risk classification such as low risk, medium risk, and high risk.
6. Include additional financial conditions such as debt-to-income ratio.
7. Develop a graphical user interface.
8. Connect the system to a database containing a larger number of customer records.
9. Add more detailed explanations for rejected applications.
10. Extend the system into a more comprehensive banking decision-support system.

20. Conclusion

This project successfully developed a **Prolog-Based Loan Pre-Screening Expert System** for the banking domain.

The system represents customer information using Prolog facts and uses production rules to evaluate loan eligibility. The implementation demonstrates important Artificial Intelligence and Prolog concepts, including knowledge representation, logical rules, forward chaining, backward chaining, unification, and backtracking.

The system was tested using five customer cases with different combinations of income, credit score, employment status, existing loans, and repayment history. The results showed that the system can apply the implemented rules and generate either an eligible or not eligible decision.

The project also includes an explanation mechanism that provides reasoning for the final decision. Although the current system is simplified, it provides a useful demonstration of how expert systems can be applied to banking decision-support problems.

Overall, the project demonstrates how Prolog can be used to develop a rule-based intelligent system for preliminary loan eligibility assessment.

References

1. S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2020.
2. D. L. Poole and A. K. Mackworth, *Artificial Intelligence: Foundations of Computational Agents*, 3rd ed. Cambridge, U.K.: Cambridge University Press, 2023.
3. M. Kubat, *Fundamentals of Artificial Intelligence: Problem Solving and Automated Reasoning*, 1st ed. New York, NY, USA: McGraw Hill, 2023.
4. V. Belle, *Toward Robots That Reason: Logic, Probability & Causal Laws*. Cham, Switzerland: Springer, 2023.
5. S. Zhang and Y. Zhang, Eds., *Artificial Intelligence Logic and Applications: The 3rd International Conference, AILA 2023, Proceedings*. Singapore: Springer, 2023.
6. C. Cao, H. Chen, L. Zhao, J. Arshad, T. Asyhari, and Y. Wang, Eds., *Knowledge Science, Engineering and Management: 17th International Conference, KSEM 2024, Proceedings, Part III*. Singapore: Springer, 2024.
7. M. Console and B. Konev, Eds., *Reasoning Web: Declarative Artificial Intelligence—Knowledge, Rules, Logic*. Cham, Switzerland: Springer, 2025.
8. B. P. Bhuyan, A. Ramdane-Cherif, T. P. Singh, and R. Tomar, *Neuro-Symbolic Artificial Intelligence: Bridging Logic and Learning*. Singapore: Springer, 2025.
9. SWI-Prolog Developers, *SWI-Prolog Reference Manual*, ver. 10.0.2, 2026.
10. SWI-Prolog Developers, "SWI-Prolog Documentation." [Online]. Available: SWI-Prolog Documentation. [Accessed: Aug. 27, 2026].

GitHub Repository

URL: https://github.com/suri307/MLA0103-AI/tree/main/CO5/CO5-AT2

EVALUATION RUBRICS:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Industry Problem Analysis and Understanding	10%	9–10% – Clearly identifies a relevant real-world industry problem, stakeholders, constraints, inputs, and expected decisions.	7–8% – Good understanding with minor omissions.	5–6% – Basic understanding with limited analysis.	0–4% – Problem is unclear, inappropriate, or poorly analyzed.
2. Domain Knowledge and Knowledge Acquisition	10%	9–10% – Acquires comprehensive and reliable domain knowledge and identifies relevant facts, entities, and relationships.	7–8% – Relevant knowledge is acquired with minor gaps.	5–6% – Basic domain knowledge with limited sources.	0–4% – Insufficient, unreliable, or inappropriate domain knowledge.
3. Knowledge Representation and Production Rules	15%	13–15% – Develops a complete, consistent, and well-structured Prolog knowledge base with appropriate facts, predicates, and production rules.	10–12% – Well-designed knowledge base with minor omissions or inconsistencies.	7–9% – Basic knowledge base with several missing or unclear rules.	0–6% – Incomplete, inconsistent, or inappropriate knowledge representation.
4. Forward and Backward Chaining	15%	13–15% – Correctly implements and clearly demonstrates both forward and backward chaining with appropriate industry scenarios and reasoning traces.	10–12% – Both mechanisms are demonstrated with minor limitations.	7–9% – Basic implementation with incomplete demonstration.	0–6% – Chaining mechanisms are missing or incorrectly implemented.
5. Prolog Implementation and Inference	15%	13–15% – Effectively uses Prolog facts, rules, predicates, unification, backtracking, and inference to generate accurate conclusions.	10–12% – Functional implementation with minor technical issues.	7–9% – Basic implementation with limited use of Prolog features.	0–6% – Implementation is incomplete, incorrect, or non-functional.
6. Industry-Based Test Cases and Results	10%	9–10% – Provides diverse and realistic test cases with accurate queries, outputs, and detailed inference	7–8% – Provides relevant test cases with mostly	5–6% – Limited test cases and basic results.	0–4% – Insufficient or inappropriate testing.

		traces.	accurate results.		
7. Analysis and Interpretation of Results	10%	9–10% – Provides insightful analysis of reasoning accuracy, consistency, rule coverage, and practical industry usefulness.	7–8% – Good analysis with minor gaps.	5–6% – Basic analysis with limited interpretation.	0–4% – Results are poorly analyzed or not interpreted.
8. Procedural and Non-Procedural Paradigm Understanding	5%	5% – Clearly explains and demonstrates the distinction between procedural and declarative/non-procedural approaches in the developed system.	4% – Good conceptual distinction with minor gaps.	2–3% – Basic understanding but limited application.	0–1% – Incorrect or missing explanation.
9. Documentation, GitHub and Technical Presentation	5%	5% – Complete, well-organized report and GitHub repository with clear code, documentation, references, and professional presentation.	4% – Good documentation and repository with minor issues.	2–3% – Basic documentation with some missing components.	0–1% – Poor documentation or missing GitHub repository.
10. Limitations, Future Enhancements and Conclusion	5%	5% – Clearly identifies practical limitations and proposes realistic, technically relevant future enhancements.	4% – Identifies relevant limitations and improvements.	2–3% – Basic discussion of limitations and future scope.	0–1% – Discussion is missing or unrealistic.
Total	100%				